PA2 — EVALUACIÓN APLICADA

Gestión dinámica de solicitudes

Informe técnico y desarrollo de las cuatro actividades

Dato	Información
Curso	Algoritmo y Estructura de Datos Basados en Inteligencia Artificial
Código	30710
Tema	Listas simples, dobles y circulares
Docente	Wildr Julio Espinoza Bravo
NRC	4134
Integrantes	1. Leonardo Cuadros 2. Jhonatan Lino 3. Anthony Fernando 4. Victor Sinche
Fecha	15 de octubre de 2026

Introducción y planteamiento del problema

El caso de evaluación describe una organización cuyo registro de solicitudes cambia de manera constante: ingresan nuevos casos, otros se eliminan o cambian de posición, y ciertos procesos requieren consultar registros hacia atrás o revisarlos repetidamente. La solución propuesta representa cada solicitud mediante un nodo enlazado y selecciona una estructura distinta según la operación de atención.

Objetivo general: modelar y explicar estructuras de listas enlazadas que administren solicitudes de atención, mostrando sus operaciones, enlaces y costos de ejecución.

Datos y supuestos empleados

- Cada solicitud contiene código único dentro de la lista simple, asunto y estado (por defecto, «Pendiente»).
- La lista simple conserva referencias a la cabeza y la cola; al final de una lista simple no circular el enlace es NULO.
- La búsqueda se hace por código y permite modificar el asunto o estado, sin cambiar el identificador.
- Las propuestas de lista doble y circular representan procesos complementarios del mismo caso, no obligatoriamente la misma instancia de lista.

Organización del desarrollo

Actividad 1: conceptos, clasificación y diagrama de cinco nodos. Actividad 2: recorrido e inserciones. Actividad 3: búsqueda, modificación, eliminación y análisis de costos. Actividad 4: propuestas de listas dobles y circulares. Al final se muestra una demostración en Python y las verificaciones efectuadas.

Actividad 1. Fundamentos y representación

Una lista enlazada es una estructura lineal dinámica formada por nodos que almacenan un dato y una o más referencias que conectan cada nodo con otros. A diferencia de un arreglo indexado, para hallar una posición arbitraria normalmente se sigue la cadena desde un extremo.

Tipo	Enlaces por nodo	Comportamiento	Aplicación al caso
Simple	siguiente	Avanza en una dirección y termina en NULO.	Orden actual de solicitudes.
Doble	anterior y siguiente	Permite desplazarse en ambos sentidos.	Revisión de expedientes, adelante y atrás.
Circular simple	siguiente; la cola vuelve a la cabeza	Repite la secuencia sin un NULO final.	Rondas continuas de supervisión.

En el ejemplo inicial usamos S101 (Constancia), S102 (Orientación), S103 (Reclamo), S104 (Actualización) y S105 (Consulta). La cabeza apunta a S101 y la cola a S105. Las flechas simbolizan las referencias siguiente.



Figura. Representación inicial de una lista simple con cinco nodos vinculados al caso.

Se elige la lista simple como punto de partida porque permite ilustrar el registro secuencial sin pagar el costo de mantener un puntero anterior en cada nodo.

Actividad 2. Recorrido e inserciones

2.1 Recorrido completo

Se parte de cabeza y se visita cada nodo hasta encontrar NULO. El puntero temporal no cambia la lista; solo se desplaza para leer los registros.

```
PROCEDIMIENTO Recorrer(cabeza)
  actual <- cabeza
  MIENTRAS actual != NULO HACER
    MOSTRAR actual.dato
    actual <- actual.siguiente
  FIN MIENTRAS
```

Salida de ejemplo: S101 → S102 → S103 → S104 → S105. El recorrido usa O(n) tiempo y O(1) memoria auxiliar.

2.2 Insertar al inicio

```
nuevo <- Nodo(S100)
nuevo.siguiente <- cabeza
cabeza <- nuevo
SI cola = NULO ENTONCES cola <- nuevo
```

Después: S100 → S101 → S102 → S103 → S104 → S105. Se actualiza la cabeza; si estaba vacía, también se inicializa la cola. El ajuste de referencias cuesta O(1).

2.3 Insertar al final

```
nuevo <- Nodo(S106)
SI cola = NULO ENTONCES cabeza <- nuevo
SINO cola.siguiente <- nuevo
cola <- nuevo
```

Después: S100 → S101 → S102 → S103 → S104 → S105 → S106. Como se mantiene el puntero cola, se conecta el nuevo nodo directamente en O(1) para la operación de enlaces.

2.4 Insertar después de un nodo dado

```
previo <- Buscar("S102")
SI previo = NULO ENTONCES DEVOLVER FALSO
nuevo <- Nodo(S102A)
nuevo.siguiente <- previo.siguiente
previo.siguiente <- nuevo
SI cola = previo ENTONCES cola <- nuevo
```

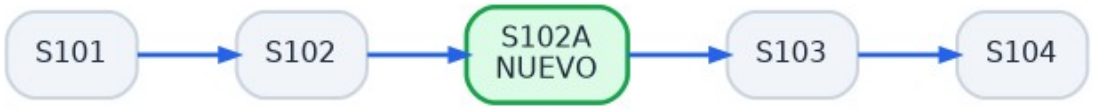


Figura. Parte de la lista después de insertar S102A a continuación de S102.

Antes: S102 → S103. Después: S102 → S102A → S103. Se conecta primero el nuevo nodo con S103 y luego S102 con el nuevo nodo, evitando perder la referencia al resto de la estructura. Encontrar S102 por código requiere O(n) en el peor caso; enlazarlo, O(1).

Actividad 3. Lista simple: búsqueda, modificación y eliminación

3.1 Búsqueda por código

Se escoge el código como criterio porque distingue solicitudes, incluso si comparten asunto. El algoritmo recorre secuencialmente hasta encontrar el código o llegar a NULO.

```
FUNCIÓN Buscar(codigo)
  actual <- cabeza
  MIENTRAS actual != NULO HACER
    SI actual.dato.codigo = codigo ENTONCES DEVOLVER actual
    actual <- actual.siguiente
  FIN MIENTRAS
  DEVOLVER NULO
```

Ejemplo: Buscar("S103") devuelve el nodo cuyo asunto es Reclamo. En el peor caso el tiempo es O(n).

3.2 Modificación de un nodo localizado

```
nodo <- Buscar("S103")
SI nodo != NULO ENTONCES
  nodo.dato.estado <- "En proceso"
FIN SI
```

Solo cambia el dato de S103; los enlaces de la lista se mantienen. La modificación de un nodo ya localizado es O(1), aunque localizarlo previamente es O(n).

3.3 Eliminación y reajuste de enlaces

```
anterior <- NULO
actual <- cabeza
MIENTRAS actual != NULO Y actual.dato.codigo != codigo HACER
  anterior <- actual
  actual <- actual.siguiente
FIN MIENTRAS
SI actual = NULO ENTONCES DEVOLVER FALSO
SI anterior = NULO ENTONCES cabeza <- actual.siguiente
SINO anterior.siguiente <- actual.siguiente
SI cola = actual ENTONCES cola <- anterior
actual.siguiente <- NULO
```

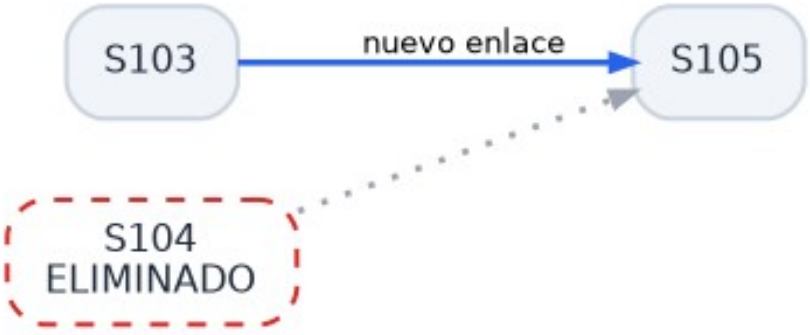


Figura. Al eliminar S104, el nodo S103 enlaza directamente con S105.

Eliminación de cabeza: cabeza pasa al segundo nodo. Eliminación interior: el anterior apunta al siguiente del eliminado. Eliminación de cola: cola pasa a su predecesor, que apunta a NULO. Cuando se elimina el único nodo, cabeza y cola quedan NULO. La operación por código necesita O(n) para localizar el nodo y su predecesor.

3.4 Reubicación: operación adicional solicitada por el caso

Se separa S105 de su posición anterior y se enlaza después de S101, sin duplicar datos. Primero se conservan las referencias y se corrige la cola si correspondiera; después se conecta S105 en su nueva posición.



Figura. Orden final de la demostración tras reubicar S105 después de S101.

3.5 Eficiencia de operaciones

Operación	Costo de punteros / recorrido	Detalle
Recorrido completo	O(n)	Visita todos los nodos.
Insertar al inicio	O(1)	Una vez validado el registro.
Insertar al final con cola	O(1)	No recorre para encontrar el final.
Insertar tras nodo conocido	O(1)	Si solo se recibe su código, buscarlo cuesta O(n).
Buscar por código	O(n)	No existe acceso directo por índice.
Modificar nodo encontrado	O(1)	Cambian datos, no enlaces.
Eliminar cabeza	O(1)	La función por código detecta la cabeza de inmediato.
Eliminar nodo interior o cola por código	O(n)	Se necesita encontrar nodo y predecesor.
Reubicar después de otro código	O(n)	Se buscan los dos nodos antes de reconectar.

Precisión de implementación: el programa Python valida que no haya códigos duplicados antes de insertar. Esa comprobación recorre la lista en O(n), por lo que insertar al inicio o al final tiene O(n) como costo completo de la llamada en este prototipo, aunque la modificación de punteros aislada sea O(1).

Actividad 4. Listas dobles y circulares

4.1 Propuesta de lista doble: consulta retrospectiva de expedientes

Situación: durante el seguimiento de solicitudes, un trabajador revisa S201, S202 y S203. Desde S203 necesita regresar a S202 y S201 para contrastar observaciones. En una lista simple debería recorrer desde cabeza para localizar el nodo previo; una lista doble guarda ambos enlaces y permite retroceder directamente desde el nodo actual.



Figura. Nodos de seguimiento con referencias anterior y siguiente.

```
nuevo.anterior <- cola
nuevo.siguiente <- NULO
SI cola = NULO ENTONCES cabeza <- nuevo
SINO cola.siguiente <- nuevo
cola <- nuevo

Recorrido inverso: actual <- cola
MIENTRAS actual != NULO HACER
   MOSTRAR actual.dato
   actual <- actual.anterior
FIN MIENTRAS
```

Ventaja: navegar al siguiente o anterior desde un nodo conocido toma O(1) por paso. Limitación: consume un enlace adicional por nodo y sus inserciones/eliminaciones deben mantener correctamente las dos referencias.

4.2 Propuesta de lista circular: ronda continua de supervisión

Situación: tres solicitudes pendientes, S301, S302 y S303, reciben revisión periódica. Tras revisar S303, el siguiente turno vuelve a S301. La lista circular modela el ciclo con el enlace cola.siguiente apuntando a la cabeza.



Figura. Lista circular simple para turnos de revisión repetidos.

```
SI cola = NULO ENTONCES
   nuevo.siguiente <- nuevo
   cola <- nuevo
SINO
   nuevo.siguiente <- cola.siguiente
   cola.siguiente <- nuevo
   cola <- nuevo
FIN SI

Para una vuelta: REPETIR tamano VECES
   MOSTRAR actual.dato
   actual <- actual.siguiente
```

Ventaja: la conexión con la cabeza permite continuar la ronda sin reiniciar manualmente. Control: no se debe esperar un nodo NULO; para una vuelta se cuentan exactamente tantos pasos como nodos existan. Los turnos de la prueba son S301 → S302 → S303 → S301 → S302.

4.3 Comparación y decisión

Necesidad	Estructura elegida	Motivo de selección
Gestionar orden, altas, bajas y cambios	Simple	Un solo enlace por nodo; suficiente si se avanza hacia adelante.
Volver al expediente previo	Doble	El enlace anterior permite retroceder sin rehacer la búsqueda.
Mantener turnos sucesivos	Circular	El último nodo vuelve al primero y preserva el ciclo.

Implementación funcional y verificación

Se construyó un prototipo en Python con las clases Solicitud, NodoSimple, ListaSimple, NodoDoble, ListaDoble, NodoCircular y ListaCircular. Las listas enlazadas se construyen con referencias entre nodos; las listas nativas de Python se emplean únicamente para reunir valores visibles durante la demostración y las pruebas.

Demostración reproducible

```
python -m app.demo
python -m unittest discover -s tests -v
```

La demostración crea cinco solicitudes, inserta al inicio/final/después de un código, busca y modifica S103, elimina S104, mueve S105 detrás de S101, muestra la navegación doble y realiza cinco turnos de la ronda circular.

Resultados de las pruebas

Conjunto	Validaciones	Resultado local
Lista simple	Recorrido; tres inserciones; búsqueda; cambio de estado; eliminación de cabeza, interior y cola; reubicación; vacía; duplicados.	11 pruebas OK
Lista doble	Recorrido hacia adelante y atrás, enlaces extremos y enlace interior.	1 prueba OK
Lista circular	Vacia, ciclo, vuelta completa y turnos repetidos.	1 prueba OK
TOTAL	Pruebas ejecutadas con unittest.	13/13 OK

Estas pruebas validan escenarios definidos y no sustituyen la revisión del docente; se pueden repetir desde la raíz del repositorio sin instalar paquetes.

Conclusiones y entrega

1. Las listas enlazadas sirven para representar una secuencia que cambia en tiempo de ejecución y exigen controlar correctamente los enlaces entre nodos.
2. Una lista simple resuelve las operaciones principales, una doble permite navegación bidireccional y una circular resulta adecuada para rondas periódicas.
3. La eficiencia depende de la operación exacta: cambiar referencias puede ser O(1), mientras que buscar un nodo por código suele ser O(n).
4. La demostración ejecutable y trece pruebas verifican las operaciones descritas, incluidos casos extremos que suelen provocar errores en los enlaces.

- Informe en PDF: el presente documento. También se incluye una versión editable en DOCX.
- Repositorio GitHub: proyecto publicado con código, diagramas, pseudocódigo, pruebas y README.
- Video: guion distribuido entre cuatro participantes; la grabación y su publicación están pendientes.
- Asistencia y seguimiento: se acreditan mediante el registro y la valoración del docente.

Repositorio:

https://github.com/totomanxd321/PA2_Listas_Enlazadas_Grupo5

Video de YouTube: pendiente de publicación.

Materiales utilizados

Enunciado y rúbrica entregados por el docente: «PA2 - Evaluación aplicada sobre listas enlazadas», curso 30710, sesiones 5 a 7 (3 páginas). Los algoritmos, diagramas y ejemplos de este informe son una elaboración del equipo propuesta para resolver el caso.